

NAME : PATOLIYA MAULIKKUMAR NARENDRABHAI

BRANCH : CE (24CEUCG121)

BATCH : A2

ROLL NO. : 041

SUB : ADVANCED C++ PROGRAMMING CONCEPTS

FOCUS: OVERLOAD RESOLUTION | COMPILER BEHAVIOR

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c2){
cout << "Inside method : ";
return Complex(this->re+c2.re , this->im+c2.im);
}
```

```
// Without constant arguments(pass by reference)
```

```
friend Complex operator+( Complex &c1, Complex &c2);
```

```
};
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c1, Complex &c2){
cout << "Inside friend : ";
return Complex(c1.re+c2.re , c1.im+c2.im);
}
```

```
int main() {
Complex c1(1.1, 1.1), c2(2.2, 2.2), c3 ;
```

```
// error : more than one operator "+" matches
these operands
c3 = c1 + c2;
```

```
c3.print();
```

```
return 0;
}
```

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// With constant arguments (pass by reference)
```

```
Complex operator+(const Complex &c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// With constant arguments(pass by reference)
```

```
friend Complex operator+(const Complex &c1,const Complex &c2);
```

```
};
```

```
// With constant arguments (pass by reference)
```

```
Complex operator+(const Complex &c1,const Complex &c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
c3 = c1 + c2;
c3.print();
```

```
return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// With constant arguments (pass by reference)
```

```
Complex operator+(const Complex &c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// With constant arguments(pass by reference)
```

```
friend Complex operator+(const Complex &c1,const Complex &c2);
```

```
};
```

```
// With constant arguments (pass by reference)
```

```
Complex operator+(const Complex &c1,const Complex &c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
c3 = c1 + c2;
c3.print();
```

```
return 0;
}
```

```
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> g++ .\Ex.cpp
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> ./a.exe
Inside method : 3.3 + j3.3
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> |
```

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// With constant arguments (pass by reference)
```

```
Complex operator+(const Complex &c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// Without constant arguments(pass by reference)
```

```
friend Complex operator+( Complex &c1, Complex &c2);
```

```
};
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c1, Complex &c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
c3 = c1 + c2;
c3.print();
```

```
return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// With constant arguments (pass by reference)
```

```
Complex operator+(const Complex &c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// Without constant arguments(pass by reference)
```

```
friend Complex operator+( Complex &c1, Complex &c2);
```

```
};
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c1, Complex &c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
c3 = c1 + c2;
c3.print();
```

```
return 0;
}
```

```
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> g++ .\Ex.cpp
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> ./a.exe
Inside friend : 3.3 + j3.3
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> █
```

➤ Why compiler call friend function instead of member function call ?

➡ Checking Friend Function :

- The friend function expects two non-const references: `Complex& c1`, `Complex& c2`.
- Since `c1` and `c2` are both modifiable (not `const`), this match is perfect.

➡ Checking Member Function

- The member function requires `const Complex&` as an argument.
- Since `c1` is calling it (`c1.operator+(c2)`), this is implicitly passed as `Complex* const` this.
- This means this (which is `Complex* const`) cannot bind to the member function, since `operator+` is not a `const` member function.

➤ Conclusion :

- The friend function is a better match due to the exact type match (`Complex&` arguments).
- The member function requires an implicit conversion (modifying this to be `const`), which makes it less preferable.

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im ;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// With constant arguments(pass by reference)
```

```
friend Complex operator+(const Complex &c1,const Complex &c2);
```

```
};
```

```
// With constant arguments (pass by reference)
```

```
Complex operator+(const Complex &c1,const Complex &c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im);
}
```

```
int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
c3 = c1 + c2;
c3.print();
```

```
return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im ;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// With constant arguments(pass by reference)
```

```
friend Complex operator+(const Complex &c1,const Complex &c2);
```

```
};
```

```
// With constant arguments (pass by reference)
```

```
Complex operator+(const Complex &c1,const Complex &c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im);
}
```

```
int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
c3 = c1 + c2;
c3.print();
```

```
return 0;
}
```

```
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> g++ .\Ex.cpp
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> ./a.exe
Inside method : 3.3 + j3.3
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> █
```

Case	Method Used	Arguments Type	Error Occurred ?	Reason for Error or Success	Output
C1	Member + Friend	Both takes non-const &	Yes	$M \rightarrow +(*this, \& c2)$ $F \rightarrow +(\& c1, \& c2)$	“ambiguity”
C2	Member + Friend	Both takes const &	No	$M \rightarrow +(*this, \text{const}\& c2)$ $F \rightarrow +(\text{const}\& c1, \text{const}\& c2)$	Inside method: 3.3 + j3.3
C3	Member + Friend	Member $\rightarrow$ const & friend $\rightarrow$ non-const &	No	$M \rightarrow +(*this, \text{const}\& c2)$ $F \rightarrow +(\& c1, \& c2)$	Inside friend: 3.3 + j3.3
C4	Member + Friend	Member $\rightarrow$ non-const & friend $\rightarrow$ const &	No	$M \rightarrow +(*this, \& c2)$ $F \rightarrow +(\text{const}\& c1, \text{const}\& c2)$	Inside method: 3.3 + j3.3

Conclusion

✓ Valid Conditions :

- ➡ If both functions takes (**const reference**) **const complex&** , works fine and call member function.
- ➡ If one function (either member or friend) takes (**const reference**) **const complex&** and the other one takes (**non-const reference**) **complex&** , there is no ambiguity error andoverloading works fine.
- ➡ If member function has constant arguments and friend doesn't have, then always call friend function .
- ➡ If friend function has constant arguments and member function doesn't have, then always call member function .
- ➡ If the friend function is not defined , the member function alone works fine.
- ➡ If the member function is not defined , the friend function alone works fine.

Invalid Conditions (Causes Error) :

- ➡ If both the member and friend function take non-const reference , the compiler cannot decide which one to call (ambiguity error).

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c2){
    cout << "Inside method : ";
    return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// Without constant arguments (pass by value)
```

```
friend Complex operator+( Complex c1, Complex c2);
```

```
};
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c1, Complex c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    c3 = c1 + c2;
    c3.print();
```

```
    return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c2){
    cout << "Inside method : ";
    return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// Without constant arguments (pass by value)
```

```
friend Complex operator+( Complex c1, Complex c2);
```

```
};
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c1, Complex c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
// error : more than one operator "+" matches
these operands
c3 = c1 + c2;
```

```
c3.print();
```

```
    return 0;
}
```

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// With constant arguments (pass by value)
Complex operator+(const Complex c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// With constant arguments (pass by value)
friend Complex operator+(const Complex c1,const Complex c2);

};
```

```
// With constant arguments (pass by value)
```

```
Complex operator+(const Complex c1,const Complex c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;

c3 = c1 + c2;
c3.print();

return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
double re;
double im;
```

```
public:
```

```
Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}
```

```
void print() {
cout << re << " + j" << im << endl;
}
```

```
// With constant arguments (pass by value)
Complex operator+(const Complex c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// With constant arguments (pass by value)
friend Complex operator+(const Complex c1,const Complex c2);

};
```

```
// With constant arguments (pass by value)
```

```
Complex operator+(const Complex c1,const Complex c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
// error : more than one operator "+" matches
these operands
c3 = c1 + c2;
```

```
c3.print();
```

```
return 0;
}
```

```

#include <iostream>
#include <cmath>

using namespace std;

class Complex {

double re;
double im;

public:

Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}

void print() {
cout << re << " + j" << im << endl;
}

// With constant arguments (pass by value)
Complex operator+(const Complex c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}

// Without constant arguments (pass by value)
friend Complex operator+( Complex c1, Complex c2);

};

```

// Without constant arguments (pass by value)

```

Complex operator+( Complex c1, Complex c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}

```

```

int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;

c3 = c1 + c2;
c3.print();

return 0;
}

```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
    // With constant arguments (pass by value)
    Complex operator+(const Complex c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
    // Without constant arguments (pass by value)
    friend Complex operator+( Complex c1, Complex c2);

};
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c1, Complex c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    // error : more than one operator "+" matches
    these operands
    c3 = c1 + c2;
```

```
    c3.print();
```

```
    return 0;
}
```

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c2){
    cout << "Inside method : ";
    return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// With constant arguments (pass by value)
```

```
friend Complex operator+(const Complex c1,const Complex c2);
```

```
};
```

```
// With constant arguments (pass by value)
```

```
Complex operator+(const Complex c1,const Complex c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    c3 = c1 + c2;
    c3.print();
```

```
    return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c2){
    cout << "Inside method : ";
    return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// With constant arguments (pass by value)
```

```
friend Complex operator+(const Complex c1,const Complex c2);
```

```
};
```

```
// With constant arguments (pass by value)
```

```
Complex operator+(const Complex c1,const Complex c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
// error : more than one operator "+" matches
these operands
c3 = c1 + c2;
```

```
c3.print();
```

```
    return 0;
}
```

Case	Method Used	Arguments Type	Error Occurred ?	Reason for Error or Success	Output
C1	Member + Friend	Both takes non-const val	Yes	$M \rightarrow +(*this, c2)$ $F \rightarrow +(c1, c2)$	“ambiguity”
C2	Member + Friend	Both takes const val	Yes	$M \rightarrow +(*this, \text{const } c2)$ $F \rightarrow +(\text{const } c1, \text{const } c2)$	“ambiguity”
C3	Member + Friend	Member $\rightarrow$ const val friend $\rightarrow$ non-const val	Yes	$M \rightarrow +(*this, \text{const } c2)$ $F \rightarrow +(c1, c2)$	“ambiguity”
C4	Member + Friend	Member $\rightarrow$ non-const val friend $\rightarrow$ const val	Yes	$M \rightarrow +(*this, c2)$ $F \rightarrow +(\text{const } c1, \text{const } c2)$	“ambiguity”

Conclusion

- ➡ If all arguments (parameters) of member and friend function are passed by value , then it gives ambiguous error.
- ➡ Because, all arguments takes value not reference , and therefor both the function work as same and compiler not able to decide that which function should be called.
- ➡ Doesn't matter that arguments are constant or not.

Comparison

pass by reference

Arguments Type	Error Occurred ?	Output
Both takes non-const &	Yes	“ambiguity”
Both takes const &	No	Inside method: 3.3 + j3.3
Member → const & friend → non-const &	No	Inside friend: 3.3 + j3.3
Member → non-const & friend → const &	No	Inside method: 3.3 + j3.3

pass by value

Arguments Type	Error Occurred ?	Output
Both takes non-const val	Yes	“ambiguity”
Both takes const val	Yes	“ambiguity”
Member → const val friend → non-const val	Yes	“ambiguity”
Member → non-const val friend → const val	Yes	“ambiguity”

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// With constant arguments (pass by reference)
```

```
    Complex operator+(const Complex &c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
// Without constant arguments (pass by value)
```

```
    friend Complex operator+( Complex c1, Complex c2);
```

```
};
```

```
// Without constant arguments (pass by value)
```

```
    Complex operator+( Complex c1, Complex c2){
        cout << "Inside friend : ";
        return Complex( c1.re + c2.re , c1.im + c2.im );
    }
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3;
```

```
    c3 = c1 + c2;
    c3.print();
```

```
    return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// With constant arguments (pass by reference)
```

```
    Complex operator+(const Complex &c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
// Without constant arguments (pass by value)
```

```
    friend Complex operator+( Complex c1, Complex c2);
```

```
};
```

```
// Without constant arguments (pass by value)
```

```
    Complex operator+( Complex c1, Complex c2){
        cout << "Inside friend : ";
        return Complex( c1.re + c2.re , c1.im + c2.im );
    }
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
// error : more than one operator "+" matches
these operands
c3 = c1 + c2;
```

```
    c3.print();
```

```
    return 0;
}
```

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
    // With constant arguments (pass by reference)
```

```
    Complex operator+(const Complex &c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
    // With constant arguments (pass by value)
```

```
    friend Complex operator+(const Complex c1,const Complex c2);
```

```
};
```

```
// With constant arguments (pass by value)
```

```
Complex operator+(const Complex c1,const Complex c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    c3 = c1 + c2;
```

```
    c3.print();
```

```
    return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
    // With constant arguments (pass by reference)
```

```
    Complex operator+(const Complex &c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
    // With constant arguments (pass by value)
```

```
    friend Complex operator+(const Complex c1,const Complex c2);
```

```
};
```

```
    // With constant arguments (pass by value)
```

```
    Complex operator+(const Complex c1,const Complex c2){
        cout << "Inside friend : ";
        return Complex( c1.re + c2.re , c1.im + c2.im );
    }
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    // error : more than one operator "+" matches
    these operands
    c3 = c1 + c2;
```

```
    c3.print();
```

```
    return 0;
}
```

```

#include <iostream>
#include <cmath>

using namespace std;

class Complex {

double re;
double im;

public:

Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}

void print() {
cout << re << " + j" << im << endl;
}

// Without constant arguments (pass by reference)
Complex operator+( Complex &c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}

// With constant arguments (pass by value)
friend Complex operator+(const Complex c1,const Complex c2);

};

```

// With constant arguments (pass by value)

```

Complex operator+(const Complex c1,const Complex c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}

```

```

int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;

```

```

c3 = c1 + c2;

```

```

c3.print();

```

```

return 0;
}

```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by reference)
```

```
    Complex operator+( Complex &c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
// With constant arguments (pass by value)
```

```
    friend Complex operator+(const Complex c1,const Complex c2);
```

```
};
```

```
// With constant arguments (pass by value)
```

```
    Complex operator+(const Complex c1,const Complex c2){
        cout << "Inside friend : ";
        return Complex( c1.re + c2.re , c1.im + c2.im );
    }
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
// error : more than one operator "+" matches
these operands
c3 = c1 + c2;
```

```
    c3.print();
```

```
    return 0;
}
```

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by value)
```

```
    Complex operator+( Complex c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
// With constant arguments (pass by reference)
```

```
    friend Complex operator+(const Complex &c1,const Complex &c2);
```

```
};
```

```
// With constant arguments (pass by reference)
```

```
    Complex operator+(const Complex &c1,const Complex &c2){
        cout << "Inside friend : ";
        return Complex( c1.re + c2.re , c1.im + c2.im );
    }
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    c3 = c1 + c2;
    c3.print();
```

```
    return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by value)
```

```
    Complex operator+( Complex c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
// With constant arguments (pass by reference)
```

```
    friend Complex operator+(const Complex &c1,const Complex &c2);
```

```
};
```

```
// With constant arguments (pass by reference)
```

```
    Complex operator+(const Complex &c1,const Complex &c2){
        cout << "Inside friend : ";
        return Complex( c1.re + c2.re , c1.im + c2.im );
    }
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    c3 = c1 + c2;
    c3.print();
```

```
    return 0;
}
```

```
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> g++ .\Ex.cpp
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> ./a.exe
Inside method : 3.3 + j3.3
PS C:\Users\Maulik N Patoliya\OneDrive\Desktop\Cpp Tutorial> 
```

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c2){
    cout << "Inside method : ";
    return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// Without constant arguments (pass by reference)
```

```
friend Complex operator+( Complex &c1, Complex &c2);
```

```
};
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c1, Complex &c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    c3 = c1 + c2;
    c3.print();
```

```
    return 0;
}
```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
// Without constant arguments (pass by value)
```

```
Complex operator+( Complex c2){
    cout << "Inside method : ";
    return Complex( this->re + c2.re , this->im + c2.im );
}
```

```
// Without constant arguments (pass by reference)
```

```
friend Complex operator+( Complex &c1, Complex &c2);
```

```
};
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c1, Complex &c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
// error : more than one operator "+" matches
these operands
c3 = c1 + c2;
```

```
c3.print();
```

```
    return 0;
}
```

```

#include <iostream>
#include <cmath>

using namespace std;

class Complex {

double re;
double im;

public:

Complex(double re = 0, double im = 0) {
this->re = re;
this->im = im;
}

void print() {
cout << re << " + j" << im << endl;
}

// With constant arguments (pass by value)
Complex operator+(const Complex c2){
cout << "Inside method : ";
return Complex( this->re + c2.re , this->im + c2.im );
}

// Without constant arguments (pass by reference)
friend Complex operator+( Complex &c1, Complex &c2);

};

```

// Without constant arguments (pass by reference)

```

Complex operator+( Complex &c1, Complex &c2){
cout << "Inside friend : ";
return Complex( c1.re + c2.re , c1.im + c2.im );
}

```

```

int main() {
Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;

c3 = c1 + c2;
c3.print();

return 0;
}

```

output ?

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
class Complex {
```

```
    double re;
    double im;
```

```
public:
```

```
    Complex(double re = 0, double im = 0) {
        this->re = re;
        this->im = im;
    }
```

```
    void print() {
        cout << re << " + j" << im << endl;
    }
```

```
    // With constant arguments (pass by value)
    Complex operator+(const Complex c2){
        cout << "Inside method : ";
        return Complex( this->re + c2.re , this->im + c2.im );
    }
```

```
    // Without constant arguments (pass by reference)
    friend Complex operator+( Complex &c1, Complex &c2);

};
```

```
// Without constant arguments (pass by reference)
```

```
Complex operator+( Complex &c1, Complex &c2){
    cout << "Inside friend : ";
    return Complex( c1.re + c2.re , c1.im + c2.im );
}
```

```
int main() {
    Complex c1(1.1, 1.1) , c2(2.2, 2.2) , c3 ;
```

```
    // error : more than one operator "+" matches
    these operands
    c3 = c1 + c2;
```

```
    c3.print();
```

```
    return 0;
}
```

Conclusion

- ➡ If arguments of member function is pass by reference and friend function are pass by value , then doesn't matter that arguments are constant or not , and generate ambiguity error.
- ➡ If member argument is pass by value and friend arguments are pass reference (const and non-const) generate ambiguity error. (Q-9 to Q-14 ; Except Q-12)

➤ Unique Case :

- ➡ Member function takes (non-const) pass by value argument and friend function takes constant reference , at that time code will compile fine and give output with calling member function. (Q - 12)

Case	Method Used	Arguments Type	Error Occurred ?	Reason for Error or Success	Output
C1	Member + Friend	Member → const & friend → non-const val	Yes	M → +(*this, const& c2) F → +(c1 , c2)	“ambiguity”
C2	Member + Friend	Member → const & friend → const val	Yes	M → +(*this , const& c2) F → +(const c1 , const c2)	“ambiguity”
C3	Member + Friend	Member → non-const & friend → const val	Yes	M → +(*this , & c2) F → +(const c1 ,const c2)	“ambiguity”

Case	Method Used	Arguments Type	Error Occurred ?	Reason for Error or Success	Output
C4	Member + Friend	Member → non-const val friend → const &	No	M → +(*this, c2) F → +(const& c1 , const& c2)	Inside method: 3.3 + j3.3
C5	Member + Friend	Member → non-const val friend → non-const &	Yes	M → +(*this , c2) F → +(& c1 , & c2)	“ambiguity”
C6	Member + Friend	Member → const val friend → non-const &	Yes	M → +(*this , const c2) F → +(& c1 ,& c2)	“ambiguity”

➤ Final Conclusion :

- ➡ The best approach is to define either a member or a friend operator+, not both. Use const references in parameter(s) to maximize compatibility and avoid ambiguity.
- ➡ If you want to define both , use const references in both member and friend functions to avoid ambiguity and ensure correct function selection.

// Member function (const reference)

Complex operator+(const Complex& c2);

// Friend function (const reference)

friend Complex operator+(const Complex& c1, const Complex& c2);

- ➡ This ensures that the operator can be used correctly without ambiguity.